Architecture 2.0: The XR Lighthouse Workshop

Translating AI-Assisted Principles into Verifiable Hardware Practice

Objective: Navigate the physical constraints, verification bottlenecks, and proprietary data moats of AI-assisted chip design through 12 interactive Colab notebooks.

The XR Lighthouse Benchmark

We are building a low-power mobile XR subsystem.

This is our *SWE-bench for hardware*. It imposes strict constraints:

- 64-bit RISC-V compute core
- 3W thermal budget
- 15ms latency limit
- Tight die area constraints

The Challenge: Static dictionaries are not benchmarks. You will interact with a parameterized mathematical model where changing the L2 cache size has **non-linear** secondary effects on off-chip traffic, power, and latency.

IP Provenance and Viral Contamination

Lab 01: The Danger of Raw LLM Regurgitation

When LLMs output raw hardware code (e.g., Verilog/Chisel), they often regurgitate snippets from their training data.

- **GPLv3 / Viral Licenses:** Legally compromises the entire proprietary SoC.
- **Unknown Provenance:** Presents a critical risk for tapeout compliance.
- **The Solution:** We must enforce strict context wiping and *IP Isolation* when transferring generated artifacts to proprietary physical validation.

Navigating Proprietary Data Moats

Lab 02: Open-Source Agility vs. Physical Realities

Why do AI-generated architectures frequently fail in the final mile?

- **Open-Source Tools (e.g., SCALE-Sim):** Fast, mathematically deterministic, great for high-volume upstream screening.
- **The Data Moat:** Closely guarded physical PPA libraries (Power, Performance, Area) held by incumbent foundries.

The Gap: A design deemed "optimal" by a linear open-source estimator will catastrophically fail at the proprietary foundry due to non-linear physical constraints (e.g., wire delay congestion limits).

Surviving the Verification Bottleneck

Lab 05: Structural Pruning Proxies

Flooding a narrow, expensive EDA pipeline with cheap AI-generated candidates leads to O(N) verification explosions.

The Fix: We build an imperfect but fast heuristic proxy that prunes failing heuristics in milliseconds, saving hours of simulator time.

Workshop Execution

Let’s dive in.

- Open Google Colab.
- Navigate to `Lab_00_Introduction_and_Setup.ipynb`.
- Follow the 12-lab sequence to evaluate, design, verify, and legally secure a hardware architecture.

Ready to start.